

Week 4 — Session 5

Topic: Strassen's Matrix Multiplication

Language: C++17

Focus: divide-and-conquer matrix multiplication, prefix sums, and matrix layer manipulation.

Task 1: Strassen's Matrix Multiplication

Practice: <https://www.geeksforgeeks.org/problems/multiply-the-matrices-1587115620/>

Educational implementation of Strassen's divide-and-conquer matrix multiplication for square matrices whose size is a power of two. The algorithm uses seven recursive multiplications instead of eight.

Solution Code

```
#include <bits/stdc++.h>
using namespace std;

using Matrix = vector<vector<long long>>>;

Matrix add(const Matrix& A, const Matrix& B) {
    int n = A.size();
    Matrix C(n, vector<long long>(n));

    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            C[i][j] = A[i][j] + B[i][j];

    return C;
}

Matrix subtractMatrix(const Matrix& A, const Matrix& B) {
    int n = A.size();
    Matrix C(n, vector<long long>(n));

    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            C[i][j] = A[i][j] - B[i][j];

    return C;
}

Matrix strassen(const Matrix& A, const Matrix& B) {
    int n = A.size();

    if (n == 1)
        return {{A[0][0] * B[0][0]}};

    int k = n / 2;

    Matrix A11(k, vector<long long>(k));
    Matrix A12(k, vector<long long>(k));
    Matrix A21(k, vector<long long>(k));
    Matrix A22(k, vector<long long>(k));

    Matrix B11(k, vector<long long>(k));
    Matrix B12(k, vector<long long>(k));
    Matrix B21(k, vector<long long>(k));
    Matrix B22(k, vector<long long>(k));

    for (int i = 0; i < k; ++i) {
        for (int j = 0; j < k; ++j) {
            A11[i][j] = A[i][j];
            A12[i][j] = A[i][j + k];
            A21[i][j] = A[i + k][j];
            A22[i][j] = A[i + k][j + k];

            B11[i][j] = B[i][j];
            B12[i][j] = B[i][j + k];
            B21[i][j] = B[i + k][j];
            B22[i][j] = B[i + k][j + k];
        }
    }

    Matrix M1 = strassen(add(A11, A22), add(B11, B22));
    Matrix M2 = strassen(add(A21, A22), B11);
    Matrix M3 = strassen(A11, subtractMatrix(B12, B22));
    Matrix M4 = strassen(A22, subtractMatrix(B21, B11));
    Matrix M5 = strassen(add(A11, A12), B22);
    Matrix M6 = strassen(subtractMatrix(A21, A11), add(B11, B12));
    Matrix M7 = strassen(subtractMatrix(A12, A22), add(B21, B22));
```

```

Matrix C11 = add(subtractMatrix(add(M1, M4), M5), M7);
Matrix C12 = add(M3, M5);
Matrix C21 = add(M2, M4);
Matrix C22 = add(subtractMatrix(add(M1, M3), M2), M6);

Matrix C(n, vector<long long>(n));

for (int i = 0; i < k; ++i) {
    for (int j = 0; j < k; ++j) {
        C[i][j] = C11[i][j];
        C[i][j + k] = C12[i][j];
        C[i + k][j] = C21[i][j];
        C[i + k][j + k] = C22[i][j];
    }
}

return C;
}

int main() {
    int n;
    cin >> n;

    Matrix A(n, vector<long long>(n));
    Matrix B(n, vector<long long>(n));

    for (auto& row : A)
        for (auto& x : row)
            cin >> x;

    for (auto& row : B)
        for (auto& x : row)
            cin >> x;

    Matrix C = strassen(A, B);

    for (const auto& row : C) {
        for (long long x : row)
            cout << x << ' ';
        cout << '\n';
    }

    return 0;
}

```

Task 2: Matrix Block Sum

Practice: <https://leetcode.com/problems/matrix-block-sum/>

For every cell, calculate the sum of all matrix elements within Manhattan row/column distance k.

Solution Code

```

#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> matrixBlockSum(
    const vector<vector<int>>& mat, int k) {

    int m = mat.size();
    int n = mat[0].size();

    vector<vector<int>> prefix(m + 1, vector<int>(n + 1, 0));

    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            prefix[i + 1][j + 1] =
                mat[i][j]
                + prefix[i][j + 1]
                + prefix[i + 1][j]
                - prefix[i][j];
        }
    }

    vector<vector<int>> ans(m, vector<int>(n));

    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            int r1 = max(0, i - k);
            int c1 = max(0, j - k);
            int r2 = min(m - 1, i + k);
            int c2 = min(n - 1, j + k);

            ans[i][j] =
                prefix[r2 + 1][c2 + 1]
                - prefix[r1][c2 + 1]
                - prefix[r2 + 1][c1]
                + prefix[r1][c1];
        }
    }

    return ans;
}

```

```

    }
}

return ans;
}

int main() {
    int m, n, k;
    cin >> m >> n >> k;

    vector<vector<int>> mat(m, vector<int>(n));

    for (auto& row : mat)
        for (int &x : row)
            cin >> x;

    auto ans = matrixBlockSum(mat, k);

    for (const auto& row : ans) {
        for (int x : row)
            cout << x << ' ';
        cout << '\n';
    }

    return 0;
}

```

Task 3: Matrix Layer Rotation

Practice: <https://www.hackerrank.com/challenges/matrix-rotation-algo/>

Rotate each layer (ring) of a matrix anti-clockwise by r positions.

Solution Code

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int m, n, r;
    cin >> m >> n >> r;

    vector<vector<int>> a(m, vector<int>(n));

    for (auto& row : a)
        for (int &x : row)
            cin >> x;

    int layers = min(m, n) / 2;

    for (int layer = 0; layer < layers; ++layer) {
        vector<pair<int,int>> pos;

        // Top row: left -> right
        for (int j = layer; j < n - layer; ++j)
            pos.push_back({layer, j});

        // Right column: top -> bottom
        for (int i = layer + 1; i < m - layer; ++i)
            pos.push_back({i, n - layer - 1});

        // Bottom row: right -> left
        for (int j = n - layer - 2; j >= layer; --j)
            pos.push_back({m - layer - 1, j});

        // Left column: bottom -> top
        for (int i = m - layer - 2; i > layer; --i)
            pos.push_back({i, layer});

        vector<int> values;
        for (auto [i, j] : pos)
            values.push_back(a[i][j]);

        int len = values.size();
        int shift = r % len;

        for (int idx = 0; idx < len; ++idx) {
            auto [i, j] = pos[idx];
            a[i][j] = values[(idx + shift) % len];
        }
    }

    for (const auto& row : a) {
        for (int x : row)
            cout << x << ' ';
    }
}

```

```
        cout << '\n';  
    }  
    return 0;  
}
```

All programs are standalone C++17 solutions and can be compiled with a standard C++17 compiler.